Either & Railway-Oriented Programming

Either/Railway · Validate user id pipeline without exceptions · Build with Waqar

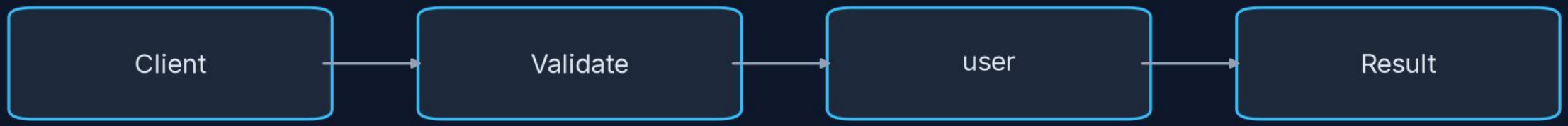
Today's Hook

- Problem: Validate user id pipeline without exceptions
- Pattern: Either/Railway — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Validate user id pipeline without exceptions
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Either/Railway: Validate user id pipeline without exceptions

Real-World Scenario

- Running example: Validate user id pipeline without exceptions
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.functional.either.problem;

import java.util.*;

public final class NaiveUser {
  public static String load(String id){ if(id==null) throw new RuntimeException("null"); return "user:"+id; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.functional.either.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

sealed interface Either<L,R> permits Left, Right { boolean isRight(); L left(); R right(); }
record Left<L,R>(L value) implements Either<L,R> { public boolean isRight(){return false;} public L left(){return value;} public R right(){throw new IllegalStateException();} }
record Right<L,R>(R value) implements Either<L,R> { public boolean isRight(){return true;} public L left(){throw new IllegalStateException();} public R right(){return value;} }
final class UserPipeline {
  static Either<String,Integer> parseInt(String s){ try { return new Right<>(Integer.parseInt(s)); } catch(Exception e){ return new Left<>("parse"); } }
  static Either<String,Integer> requirePositive(int n){ return n>0? new Right<>(n): new Left<>("positive"); }
  static Either<String,String> label(int n){ return new Right<>("user:"+n); }
  static Either<String,String> run(String raw){
    var a=parseInt(raw); if(!a.isRight()) return new Left<>(a.left());
    var b=requirePositive(a.right()); if(!b.isRight()) return new Left<>(b.left());
    return label(b.right());
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.either.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Either: user id railway ===");
    System.out.println("--- naive ---");
try { System.out.println(NaiveUser.load(null)); } catch(Exception e){ System.out.println("caught"); }
System.out.println("--- either ---");
System.out.println(UserPipeline.run("42"));
System.out.println(UserPipeline.run("x"));
System.out.println(UserPipeline.run("-1"));
  }
}
```

Walkthrough Output

```
=== Either: user id railway ===
--- naive ---
caught
--- either ---
Right[value=user:42]
Left[value=parse]
Left[value=positive]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Either/Railway.

Related Patterns

- Contrast Either/Railway with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Either/Railway → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.